

108 Agile — Playbook: Agile & DevOps per PMI Italiane

Manuale interno — Elios Scoglio

Software & Architecture Manager | Fractional CTO | NLP Counselor certificato

Versione 1.0 — Maggio 2026

Questo è il manuale operativo per erogare il servizio di consulenza Agile & DevOps per PMI italiane. Non è teoria. Non è marketing. È quello che funziona sul campo, scritto per me stesso. Ogni sezione corrisponde a un momento reale dell'engagement: assessment, decisione del metodo, adozione, DevOps, metriche, resistenza, template.

Indice

- Parte 1 — Agile Maturity Assessment (1 giorno, €5.000)
 - Parte 2 — Scrum vs Kanban vs Ibrido: la decisione per PMI italiane
 - Parte 3 — 90-Day Agile Adoption Program (€4.000–€8.000/mese)
 - Parte 4 — DevOps Foundation per PMI (senza budget enterprise)
 - Parte 5 — DORA Metrics per Team Piccoli
 - Parte 6 — Gestire la Resistenza e le Obiezioni
 - Parte 7 — Template e Strumenti
 - Parte 8 — Upsell e Continuità
-

PARTE 1 — Agile Maturity Assessment

Prodotto: Agile Maturity Assessment

Prezzo: €5.000 (1 giorno + report consegnato entro 48 ore)

Output: Executive Summary (1 pagina) + Report tecnico (10-15 pagine) + Scorecard + Raccomandazioni priorizzate

1.1 Protocollo di assessment in 1 giornata

L'assessment si svolge in sede (preferibile) o in remoto (Zoom con camera obbligatoria per i colloqui individuali). La giornata è strutturata. Non improvvisare: il protocollo serve a raccogliere dati comparabili e a mantenere l'autorevolezza nel processo.

Struttura della giornata (8 ore)

08:30 — Kick-off con il responsabile (30 min)

Chi deve esserci: titolare o CEO, eventuale CTO/IT Manager.

Obiettivo: allineare le aspettative, spiegare cosa succederà nella giornata, raccogliere i dati di contesto.

Domande da fare in questa fase:

- "Cosa vi ha spinto a chiamarmi adesso, e non 6 mesi fa?"
- "Qual è il vostro problema principale in questo momento: ritardi, qualità, comunicazione, o tutt'e tre?"
- "C'è qualcosa che non vuole che io scopra?" — questa domanda, fatta con il sorriso, rompe il ghiaccio e spesso ottieni la verità.
- "Se tra 3 mesi doveste dire che questo ha funzionato, cosa sarebbe diverso?"

Raccogliere: numero team, struttura (team unico, team multipli, full-remote, ibrido, on-site), stack tecnologico, clienti principali, tipo di prodotto (SaaS, custom, interno), presenza o assenza di backlog, strumenti attuali.

09:00 — Colloquio con il team leader/SM/PM (45 min)

Chi deve esserci: chi coordina il lavoro quotidiano (anche se il ruolo non si chiama Scrum Master).

Domande chiave:

- "Come si decide cosa fare nello sprint/settimana?"
- "Chi parla con i clienti? Con quale frequenza?"
- "Quanto tempo passa dal 'vorremmo fare X' al 'X è in produzione'?"
- "Quando qualcosa va storto, come lo scoprite? Prima o dopo il cliente?"
- "Qual è la cosa che ti fa perdere più tempo ogni settimana?"
- "Le stime vengono rispettate? Se no, perché secondo te?"

Nota: questa persona è la più sincera se si sente ascoltata e non giudicata. Usa il rapport NLP: pacing sulla postura, matching del ritmo vocale, riformulazione delle risposte. Non correggere, non interrompere con giudizi.

09:45 — Colloqui individuali con i developer (20 min ciascuno, max 4 persone)

Fai girare i developer uno alla volta. Questo dà risposte non allineate alla versione "ufficiale".

Domande per ogni developer:

- "Quanto spesso ricevi requisiti chiari quando inizi una task?"
- "Hai mai finito una feature e poi scoperto che non serviva più, o che era sbagliata?"
- "Quando hai un dubbio su cosa fare, cosa succede?"
- "Quanto è facile fare un deploy in produzione? Quante persone ci vogliono?"
- "C'è qualcosa che cambia ogni settimana senza preavviso?"
- "Se potessi cambiare una cosa sola del modo in cui lavorate, cosa sarebbe?"

Annotare le risposte verbatim. Le parole scelte rivelano il modello mentale. "Caos", "sempre", "mai", "nessuno ci dice niente", "dipende da X" sono segnali.

11:30 — Analisi degli artefatti (60 min)

Cosa guardare, in ordine di priorità:

- Backlog: esiste? È ordinato per priorità? Le card hanno descrizione leggibile? Le acceptance criteria ci sono?
- Burndown/Velocità: c'è storico? I valori sono stabili o erratici?
- Definizione di Done: esiste un documento o una checklist? Dove vive?
- Pipeline CI/CD: c'è? Quanto tempo ci vuole per un deploy? Quante volte si rompe a settimana?
- Retrospective: ci sono note delle ultime 3? Cosa hanno prodotto?
- Incident log: ci sono tracking degli errori in produzione? Tempo medio di risoluzione?
- Roadmap: esiste? È comunicata al team?

Per ogni artefatto: fotografare o screenshot. Non giudicare in questa fase. Raccogliere.

12:30 — Pranzo (solo se il cliente lo propone, altrimenti pausa libera)

13:30 — Workshop con tutto il team (90 min)

Questo è il momento più ricco. Riunisci tutti (team + titolare/PM) e fai una sessione facilitata.

Attività 1 — "Value Stream Mapping" semplificata (45 min):

Disegna sulla lavagna (fisica o Miro) il flusso di una feature: dall'idea al deploy. Chiedi al team di stimare il tempo in ogni step. Questo rivela i colli di bottiglia in modo visivo e partecipato.

Passaggi tipici: Idea → Requisito scritto → Stima → Sprint planning → In sviluppo → Code review → Test → Deploy staging → Test QA → Deploy prod

Chiedi per ogni passaggio:

- Quanto dura di media?
- Quante volte si torna indietro?
- Chi è il decision maker?

Attività 2 — "Dot voting sulle frizioni" (30 min):

Scrivi su post-it (o slide) le frizioni emerse durante la mattina. Ogni persona ha 3 voti. Votano le frizioni che percepiscono come più impattanti. Questo ordina le priorità in modo democratico e non contenzioso.

Attività 3 — Wrap-up (15 min):

Ringrazia il team. Di cosa succede dopo (report entro 48 ore, call di presentazione). Non anticipare le conclusioni.

15:00 — Completamento Scorecard (60 min, in autonomia)

Dopo il workshop compilo la scorecard. Non farlo in presenza del cliente per evitare interferenze.

16:00 — Call finale con il titolare (30 min)

Non è la presentazione dei risultati. È la raccolta delle aspettative finali:

- "Cosa vi aspettate dal report?"
- "C'è qualcosa che abbiamo toccato oggi su cui volete profondità maggiore?"
- Conferma orario e formato della presentazione dei risultati (solitamente 48-72 ore dopo).

1.2 I 5 livelli di maturità Agile

Questi livelli sono il riferimento per posizionare il cliente nella scorecard e comunicare la direzione di miglioramento.

Livello 1 — Caotico

Il team non ha processo definito. Tutto dipende da una o due persone chiave. I requisiti cambiano quotidianamente senza comunicazione formale. Non esiste backlog prioritizzato. Il deploy è manuale e rischioso. Non ci sono meeting cadenzati o, se ci sono, non producono nulla di concreto. Il codice va in produzione quando "sembra ok".

Segnali caratteristici:

- "Noi non facciamo Agile, facciamo le cose come vengono"
- Nessuna definizione di Done
- Bug in produzione scoperti dal cliente prima del team
- Nessuna stima o stime completamente inattendibili

Cosa dire al cliente: "Siete in una fase normale per molte aziende. Non è colpa di nessuno. Ma ha un costo misurabile in tempo perso, clienti insoddisfatti e stress del team. Il potenziale di miglioramento è il più alto possibile."

Livello 2 — Iniziale

Il team ha adottato alcune pratiche Agile, ma in modo parziale o incoerente. C'è un backlog, ma non sempre aggiornato. Le riunioni ci sono ma spesso sfiorano o vengono saltate. Il deploy è semi-automatizzato. La Definition of Done esiste ma non viene rispettata sistematicamente. La comunicazione con il cliente/PO è irregolare.

Segnali caratteristici:

- "Facciamo Scrum, più o meno"
- Sprint planning dura 3 ore invece di 2
- Le retrospettive producono azioni che poi non vengono seguite
- Il backlog ha 200 item non prioritizzati

Cosa dire al cliente: "Avete messo le fondamenta. Il problema è la consistenza. Le pratiche ci sono, ma non sono ancora un'abitudine del team. Con 90 giorni di coaching strutturato si passa al livello 3."

Livello 3 — Definito

Il processo è documentato e rispettato. Le cerimonie avvengono regolarmente. Il DoD è condiviso. La velocità è misurabile anche se non ancora stabile. Il deploy è automatizzato almeno parzialmente. C'è comunicazione regolare con il PO/cliente. Le retrospettive producono azioni tracciate.

Segnali caratteristici:

- Velocità calcolabile con varianza < 30%
- Le cerimonie rispettano i timebox
- Il backlog è pulito e prioritizzato settimanalmente
- Il team sa cos'è in scope per lo sprint corrente

Cosa dire al cliente: "Siete in una posizione solida. Il prossimo passo è rendere i dati actionable e ottimizzare i punti di

attrito."

Livello 4 — Misurabile

Il team usa le metriche per prendere decisioni. DORA metrics tracciate. Cycle time e lead time monitorati. La velocità è stabile e usata per le previsioni. Il deployment frequency è alta (almeno settimanale). Esiste una cultura del feedback continuo. Le retrospettive producono miglioramenti misurabili.

Segnali caratteristici:

- "Sappiamo che la nostra velocity media è X con deviazione standard Y"
- Le previsioni di consegna sono accurate entro 20%
- I bug in produzione sono tracciati e il trend è decrescente
- Il team propone autonomamente esperimenti di miglioramento

Cosa dire al cliente: "Siete sopra la media del mercato italiano. Il focus ora è ottimizzare, non correggere."

Livello 5 — Ottimizzante

Il processo è posseduto dal team, non dal consulente. Il team sperimenta autonomamente variazioni del processo. Le metriche guidano le decisioni di product. Il deployment è on-demand. Il feedback loop con il cliente è continuo. La cultura è di miglioramento sistemico, non di risposta reattiva ai problemi.

Segnali caratteristici:

- Il team modifica il proprio processo senza bisogno di permesso
- Le cerimonie vengono adattate in base alle retrospettive
- C'è sperimentazione strutturata (A/B test, feature flag, canary release)
- Il cliente/PO è parte integrante del processo, non un osservatore esterno

Cosa dire al cliente: "Il mio ruolo qui è limitato. Possiamo lavorare su ottimizzazioni specifiche o su scaling."

1.3 Scorecard 30 item su 6 dimensioni

Ogni item va da 0 a 3:

- 0 = Non presente o sistematicamente disatteso
- 1 = Presente ma incoerente
- 2 = Presente e generalmente rispettato
- 3 = Consolidato, team-owned, misurabile

Punteggio totale: 0-90. Livello di maturità: 0-18 = Caotico, 19-36 = Iniziale, 37-54 = Definito, 55-72 = Misurabile, 73-90 = Ottimizzante.

Dimensione 1 — Pianificazione (max 15 punti)

| Item | Punteggio (0-3)

- 1 | Il backlog è visibile, ordinato e aggiornato almeno settimanalmente
- 2 | Ogni item ha criteri di accettazione espliciti prima che entri nello sprint
- 3 | Lo Sprint Goal è definito, comunicato e condiviso dal team
- 4 | La capacità del team viene misurata e usata in fase di planning
- 5 | Le stime sono fatte dal team (non impostate dall'alto)

Dimensione 2 — Delivery (max 15 punti)

| Item | Punteggio (0-3)

- 6 | Il team consegna un incremento funzionante ad ogni sprint
- 7 | La Definition of Done è scritta, condivisa e rispettata
- 8 | Il cycle time (dall'inizio al deploy) è misurato
- 9 | Il lead time (dalla richiesta al deploy) è misurato
- 10 | I requisiti non cambiano durante lo sprint (o ci sono regole chiare per i cambi urgenti)

Dimensione 3 — Qualità (max 15 punti)**# | Item | Punteggio (0-3)**

- 11 | Esiste una suite di test automatici (almeno unit test su logica critica)
- 12 | I bug in produzione sono tracciati con trend nel tempo
- 13 | Il debito tecnico è visibile nel backlog e viene allocato capacity ogni sprint
- 14 | Le code review sono sistematiche e producono feedback costruttivo
- 15 | La qualità viene verificata prima del deploy (non solo dopo il bug report)

Dimensione 4 — Collaborazione (max 15 punti)**# | Item | Punteggio (0-3)**

- 16 | Il PO/cliente partecipa alla Sprint Review e dà feedback sul prodotto
- 17 | Il Daily Scrum avviene regolarmente e produce azioni concrete sugli impedimenti
- 18 | Il team si auto-organizza senza bisogno di assegnazione top-down delle task
- 19 | La comunicazione interna è asincrona-first con canali definiti (no tutto su WhatsApp)
- 20 | Gli impedimenti vengono rimossi entro 24-48 ore, non lasciati aperti per settimane

Dimensione 5 — DevOps (max 15 punti)**# | Item | Punteggio (0-3)**

- 21 | Esiste una pipeline CI automatizzata che esegue test ad ogni push
- 22 | Il deploy in produzione è automatizzato (o semi-automatizzato)
- 23 | Il deployment frequency è almeno settimanale
- 24 | C'è monitoring in produzione (almeno uptime + error rate)
- 25 | Il rollback è possibile in meno di 30 minuti

Dimensione 6 — Cultura (max 15 punti)**# | Item | Punteggio (0-3)**

- 26 | Le retrospettive producono azioni tracciate e verificate nello sprint successivo
- 27 | Il team celebra i successi e analizza i fallimenti senza blame
- 28 | Il management non bypassa il processo (es. inserisce task nello sprint senza planning)
- 29 | C'è una cultura di sperimentazione: il team può proporre variazioni al processo
- 30 | I miglioramenti sono visibili nel tempo (prima/dopo misurabile)

1.4 Come presentare i risultati al cliente**Format della call di presentazione (60 min)**

Struttura consigliata:

Minuti 1-10 — Contesto e metodo

Spiega brevemente come è stata condotta l'analisi. Non annoiare con la metodologia. L'obiettivo è dare credibilità al processo.

Minuti 10-25 — Executive Summary

Mostra il punteggio totale, il livello di maturità, e le 3 osservazioni principali. Usa il linguaggio del business, non quello tecnico. "Il vostro time-to-market medio è X settimane. Il benchmark per aziende come la vostra è Y. Questo gap vi costa Z opportunità al mese."

Minuti 25-45 — Dettaglio per dimensione

Mostra la scorecard dimensione per dimensione. Non è una seduta di critica. È un piano di navigazione. Usa il framing: "Su Pianificazione siete a 8/15 — questo è il punto di leva più alto perché..."

Minuti 45-55 — Raccomandazioni priorizzate

Massimo 5 raccomandazioni, ordinate per impatto/sforzo. Non elenco della spesa. Piano realistico.

Minuti 55-60 — Prossimi passi

Qui posizioni il 90-Day Program o il workshop successivo. Non vendere in modo aggressivo. Chiedi: "Sulla base di quello che avete visto, qual è la cosa che volete affrontare per prima?"

Regole di comunicazione NLP per la presentazione

Non dire "avete sbagliato" — dire "c'è un'opportunità di miglioramento"

Non dire "non fate Agile" — dire "state usando alcune pratiche Agile, e c'è spazio per renderle più efficaci"

Non nominare singole persone come problemi — sempre il sistema, mai le persone

Quando mostri un dato negativo, accoppialo sempre con il potenziale: "La velocità è erratica al 40% di varianza — se la stabilizziamo al 15%, le vostre previsioni di consegna diventano 3x più accurate"

1.5 Template report Agile Maturity Assessment

AGILE MATURITY ASSESSMENT

Cliente: [Nome Azienda]
Data: [Data]
Consulente: Elios Scoglio

EXECUTIVE SUMMARY (1 pagina)

Livello di maturità attuale: [Livello] ([Punteggio]/90)
Punti di forza principali: [3 bullet]
Aree di miglioramento prioritarie: [3 bullet]
Impatto stimato di un programma 90 giorni: [quantificato dove possibile]
Raccomandazione immediata: [1 azione concreta]

SCORECARD PER DIMENSIONE

[Tabella con punteggi per dimensione e item]

ANALISI DETTAGLIATA

- Per ogni dimensione:
- Osservazione (cosa abbiamo visto)
 - Impatto (cosa comporta oggi)
 - Raccomandazione (cosa fare)
 - Quick win (cosa fare questa settimana)

ROADMAP DI MIGLIORAMENTO

Mese 1 — Foundation: [obiettivi e azioni]
Mese 2 — Stabilization: [obiettivi e azioni]
Mese 3 — Optimization: [obiettivi e azioni]

APPENDICE

- Note dai colloqui (anonimizzate)
- Value Stream Map risultante
- Lista impedimenti identificati
- Tool assessment

PARTE 2 — Scrum vs Kanban vs Ibrido: la decisione per PMI italiana

2.1 Albero decisionale

Non esiste la scelta "giusta" in astratto. Esiste la scelta giusta per quel contesto. Usare questo albero per guidare la conversazione con il cliente, non per imporgli una risposta.



2.2 Casi tipici PMI italiane

Caso A — Software house piccola (3-6 persone, prodotto custom per clienti)

Situazione tipica: una piccola agenzia o software house che sviluppa su commessa. Il cliente cambia idea spesso. Non c'è un PO interno. Il founder fa da PM, commerciale e a volte da developer.

Raccomandazione: Scrum ibrido con sprint di 2 settimane e Kanban board visibile al cliente.

Perché: gli sprint danno struttura e aiutano a gestire le aspettative del cliente ("questa feature entra nello sprint del 3 giugno"). La Kanban board condivisa con il cliente riduce le email di aggiornamento.

Cosa non fare: imporre il ruolo di Product Owner al fondatore che ha già 5 altri ruoli. Meglio definire un momento fisso settimanale (30 min) in cui il fondatore prioritizza il backlog.

Prima azione concreta: creare il backlog su Jira o Linear (o anche Notion), farci inserire dal cliente le richieste pending, stimare le prime 10 card. Questo da solo vale più di qualsiasi training.

Caso B — Team IT interno di una PMI manifatturiera (4-8 persone)

Situazione tipica: azienda di produzione o distribuzione con un team IT interno che gestisce ERP, e-commerce, integrazioni. Il management non è tech. Il budget viene approvato in modo discontinuo. Le "urgenze" arrivano senza preavviso dal CEO o dai direttori di area.

Raccomandazione: Kanban con WIP limit + sprint mensile di review (non di planning).

Perché: il flusso di lavoro è imprevedibile per design. Le urgenze sono strutturali, non eccezionali. Non combattere la realtà — incanalala. Il Kanban board con WIP limit visibile al management mostra chiaramente perché l'urgenza di oggi blocca la feature di domani.

Strumento NLP per questa situazione: quando il CEO arriva con l'"urgenza assoluta", invece di opporsi o cedere senza pensare, fare questa domanda: "Certo, possiamo farlo. Cosa spostiamo per farlo entrare questa settimana?" Questa domanda trasferisce la responsabilità del trade-off dove deve stare: sul business.

Prima azione concreta: board Kanban con colonne To Do / In Progress / Review / Done + WIP limit di 3 per "In Progress". Mostrare al management il board in ogni update settimanale di 15 minuti.

Caso C — Startup in fase seed (3-5 persone, prodotto proprietario)

Situazione tipica: team che sta costruendo il prodotto da zero, ancora in fase di product-market fit. I requisiti cambiano ogni 2 settimane sulla base del feedback degli utenti. Non c'è ancora un processo.

Raccomandazione: Scrum con sprint di 1 settimana nelle prime 8 settimane, poi valutare se passare a 2 settimane.

Perché: il ciclo corto forza la prioritizzazione. Il team impara a "finire le cose" invece di avere 20 feature al 70%. La Sprint Review diventa il momento di confronto con i beta user.

Attenzione: in questa fase il maggior rischio è l'overhead. Niente cerimonie che durano più del necessario. Daily di 10 minuti, non 30. Sprint planning di 1 ora, non 3. Retrospettiva di 30 minuti.

Caso D — Team remoto (4-8 persone, fusi orari diversi — es. Italia + Europa Est)

Situazione tipica: team distribuito con 2-3 ore di sovrapposizione di orario. Comunicazione prevalentemente asincrona.

Raccomandazione: Kanban con daily asincrono (video loom o post scritto su Slack) + sincronizzazione settimanale (1 ora video call).

Perché: imporre cerimonie sincrone quotidiane in fusi orari diversi crea più stress che valore. Il Kanban board visibile a tutti risolve il 70% del bisogno di allineamento. L'1 ora settimanale si usa per retrospettiva + prioritizzazione.

Tool chiave: Linear o Jira per il board, Loom per i daily asincroni, Notion per la documentazione condivisa.

2.3 Come spiegare la scelta al cliente senza creare resistenza

Il cliente non deve sentire che gli stiamo "imponendo" un metodo. La scelta deve emergere come sua.

Tecnica NLP: riformulazione e verifica

Dopo l'assessment, non dire "dovreste fare Scrum". Dire:

"Sulla base di quello che mi avete raccontato — requisiti che cambiano ogni 2 settimane, team di 5 persone, scadenze fisse con il cliente — il pattern che funziona meglio in questi contesti è uno sprint di 2 settimane con una board condivisa con il cliente. Ve lo immaginate come potrebbe funzionare per voi?"

Questa formulazione:

- Aggancia la scelta ai dati che loro stessi hanno fornito (non è una mia opinione)
- Usa "il pattern che funziona" (oggettivizza, riduce la resistenza)
- Termina con una domanda aperta che li invita a visualizzare

Tecnica: il "già lo fate"

Molte PMI italiane resistono all'Agile perché lo vedono come "roba americana" o "per le startup". La risposta più efficace è mostrare che lo fanno già, in modo informale.

"Già adesso fate una riunione il lunedì dove decidete cosa fare durante la settimana, giusto? E il venerdì guardate cosa avete fatto? Bene — quello è già un embrione di sprint. Quello che facciamo è renderlo più efficace e misurabile."

2.4 Errori da evitare nell'applicare Scrum in Italia

Errore 1 — Imporre lo sprint backlog come gabbia

In molte PMI italiane c'è un capo che non accetta che lo sprint sia "chiuso". Se si sente dire "non possiamo aggiungere cose a sprint iniziato", sente una perdita di controllo.

Soluzione: non combattere il punto di principio. Accettare che ci siano "slot urgenza" nello sprint (max 20% della capacità). Meglio uno Scrum con un buffer urgenza che un processo rifiutato.

Errore 2 — Nominare Scrum Master una persona già sovraccarica

Lo SM non è una funzione aggiuntiva per chi ha già un lavoro pieno. In PMI piccole il ruolo di facilitatore può ruotare. Meglio uno SM rotante e presente che uno SM formale e assente.

Errore 3 — Fare la retrospettiva come un'accusa pubblica

La retrospettiva italiana tende a diventare o sfogo di frustrazioni o silenzio per non fare brutte figure. Usare format strutturati (4L, Start/Stop/Continue) e separare l'identificazione dei problemi dalla ricerca delle soluzioni.

Errore 4 — Usare punti story come unità di misura gerarchica

"Ho finito 3 punti, tu ne hai finiti 8" non deve mai diventare una gara. I punti misurano la complessità del lavoro, non la produttività della persona. Se il management inizia a usare i punti per valutare le performance, la velocity viene gonfiata artificialmente nel giro di 2 sprint.

Errore 5 — Saltare la retrospettiva perché "non c'è tempo"

La retrospettiva è la sola cerimonia obbligatoria se devi sceglierne una. È il meccanismo di auto-correzione del sistema. Senza di essa, i problemi si accumulano e il team si deresponsabilizza. Se lo sprint è stato difficile, la retrospettiva è ancora più importante.

Errore 6 — Trattare il Daily come un report al manager

Se il manager partecipa al Daily e fa domande sullo stato di avanzamento, il team smette di usarlo per risolvere

problemi e inizia a usarlo per gestire le aspettative verso l'alto. Il Daily è per il team, non per il management. Il manager può ascoltare ma non dovrebbe guidare o commentare.

PARTE 3 — 90-Day Agile Adoption Program

Prodotto: 90-Day Agile Adoption Program

Prezzo: €4.000–€8.000/mese (3 mesi). Totale €12.000–€24.000.

Differenziazione di prezzo: €4.000 per team piccolo (< 5 persone, 2 giorni/mese di presenza); €8.000 per team più grande (8+ persone, 4 giorni/mese + coaching continuativo).

Output: team autonomo al termine, processo posseduto internamente, metriche baseline stabilite.

3.1 Mese 1 — Foundation

Obiettivo: il team capisce il metodo, ha un backlog pulito, ha fatto il primo sprint, ha una Definition of Done.

Settimana 1 — Setup

Giorno 1 (presenza):

- Workshop "Agile in pratica" (3 ore): non teoria pura. Simulazione Sprint Planning → Daily → Review → Retro su un caso fittizio vicino al loro dominio. Il team deve toccare il processo con mano prima di applicarlo sul loro lavoro reale.
- Creazione backlog: prendi le richieste già in coda, scrivile in formato User Story, stima una selezione con Planning Poker.
- Definizione della DoD: workshop facilitato di 90 minuti. Ogni team deve scrivere la propria. Non importarla da template — quella che scrivono loro la rispettano.

Giorno 2 (presenza):

- Sprint Planning del primo sprint. Obiettivo: il team porta a casa la sua prima Sprint Goal scritta.
- Setup tool (Jira, Linear o anche Trello): configurare board, backlog, sprint.

Settimana 2 — Primo sprint in esecuzione

Presenza: 1 ora di check-in (da remoto).

Attività:

- Verificare che il Daily avvenga. Se non avviene, capire perché (resistenza? confusione sul format? orario sbagliato?)
- Rimuovere il primo impedimento reale. Documentarlo come case study.
- Controllare che la board sia aggiornata.

Settimana 3 — Mid-sprint check

Presenza: 30 minuti da remoto.

Attività:

- Burndown review: il team sta bruciando punti? Il ritmo è realistico?
- Se c'è scope creep (task non pianificate aggiunte), affrontarlo esplicitamente.
- Anticipare la Sprint Review: chi partecipa? Come viene mostrato il prodotto?

Settimana 4 — Fine primo sprint

Presenza: mezza giornata (Sprint Review + Retrospettiva).

Sprint Review (60 min): il team mostra il prodotto funzionante al PO/stakeholder. Non slide. Demo live o walkthrough.

Retrospettiva (60 min): formato 4L (Liked, Learned, Lacked, Longed for). Il consulente facilita ma non domina. L'obiettivo è che il team produca da solo almeno 3 azioni concrete per il prossimo sprint.

Checklist settimana 1

- ☐ Workshop "Agile in pratica" completato
- ☐ Backlog creato con almeno 20 item prioritizzati
- ☐ Definizione di Done scritta e approvata dal team
- ☐ Tool configurato e visibile a tutto il team
- ☐ Primo Sprint Planning completato con Sprint Goal
- ☐ Canale di comunicazione principale definito (Slack, Teams, altro)
- ☐ Daily orario fissato nel calendario
- ☐ PO identificato (anche se part-time)

Checklist settimana 2

- ☐ Daily avviene tutti i giorni alla stessa ora
- ☐ Board aggiornata almeno una volta al giorno
- ☐ Primo impedimento identificato e rimosso (o escalato)
- ☐ Sprint Goal ancora realistico (non si sono aggiunte 10 task extra)
- ☐ Nessuna task rimasta "In Progress" per più di 3 giorni senza aggiornamento

Checklist settimana 3

- ☐ Burndown mostra progresso coerente (non tutto accumulato all'ultimo giorno)
- ☐ La Sprint Review è pianificata con partecipanti confermati
- ☐ Il backlog è stato rivisto almeno una volta durante lo sprint (backlog refinement informale)
- ☐ Almeno un feedback del PO/cliente ricevuto durante lo sprint

Checklist settimana 4

- ☐ Sprint Review completata con demo reale del prodotto
- ☐ Retrospettiva completata con azioni tracciate
- ☐ Velocity del primo sprint calcolata e documentata
- ☐ Secondo sprint pianificato con Sprint Goal
- ☐ Eventuali task non completate riportate nel backlog con rivalutazione della priorità

3.2 Mese 2 — Stabilization

Obiettivo: il team migliora il processo in autonomia, inizia CI/CD base, le metriche DORA sono visibili.

Focus del mese

Il secondo mese è il più critico. La luna di miele del primo sprint è finita. Emergono le resistenze vere: il manager che bypassa il processo, il dev senior che fa da sé senza aggiornare la board, il PO che non prioritizza il backlog.

Presenza: 2 giorni nel mese (di cui almeno 1 presenza fisica se possibile).

Giorno 1 del mese (presence day)

Mattina: "Retrospective on the process" — non la retrospettiva sullo sprint, ma sul mese intero.

Domande guida:

- "Cos'è cambiato nel vostro modo di lavorare rispetto a prima?"
- "Qual è la cosa più fastidiosa del processo attuale?"
- "Cos'è andato meglio di quanto vi aspettavate?"

Pomeriggio: Workshop CI/CD base (4 ore). Vedi Parte 4 per il dettaglio.

Settimane 5-8 — Azioni chiave

Retrospective-driven improvements: ogni azione della retro ha un owner e una data. Al Daily di lunedì si verifica.

Introduzione delle metriche DORA base:

- Deployment frequency: quante volte deployate a settimana?
- Lead time for changes: dal commit al deploy, quanto passa?
- Change failure rate: quanti deploy richiedono hotfix entro 48 ore?
- Mean time to recovery: quanto ci vuole a ripristinare un servizio dopo un incidente?

Non serve un tool enterprise per tracciarle. Un Google Sheet con una riga per ogni deploy funziona per iniziare.

Checklist mensile (mese 2)

- ☐ La velocity è stabile ($\pm 20\%$ rispetto al mese precedente)
- ☐ Le azioni delle retrospettive vengono seguite almeno al 70%
- ☐ Pipeline CI configurata (almeno lint + unit test su ogni push)
- ☐ Deploy semi-automatizzato (anche solo script bash)
- ☐ DORA metrics baseline registrata (anche manualmente)
- ☐ Almeno un "quick win" visibile al cliente/management
- ☐ Nessuna regressione nelle cerimonie (Daily, Planning, Review, Retro avvengono)

3.3 Mese 3 — Optimization

Obiettivo: velocity stabile, tech debt gestito, processo posseduto dal team, consulente in uscita.

Il momento della "consegna"

Il terzo mese è esplicitamente di transizione. L'obiettivo non è che il team abbia bisogno del consulente per sempre. È che il team possa continuare da solo. Questa è la posizione più forte per il consulente: chi trasforma e poi esce con successo ottiene referral e credibilità molto più di chi crea dipendenza.

Presenza: 1-2 giorni nel mese + 1 call finale di chiusura.

Azioni chiave del mese 3

Tech debt allocation: riservare il 20% della capacity ogni sprint per tech debt esplicito. Il team nomina un "debt owner" che mantiene il backlog del debito.

Definition of Done upgrade: dopo 2 mesi di esperienza, il DoD viene arricchito. Di solito si aggiungono: "test automatici scritti", "code review approvata da almeno 1 persona", "deploy in staging verificato".

Velocity baseline definitiva: calcolata su 3 sprint = base per le previsioni future. Il team la usa per rispondere al management sulla timeline.

Retrospektiva "Meta": alla fine del programma, facilitare una retrospektiva sul programma stesso. "Come eravate 3 mesi fa? Come siete ora? Cosa avete imparato?" Questo cristallizza i risultati e dà materiale per il report finale.

Checklist mensile (mese 3)

- [] Il team conduce le cerimonie autonomamente (senza il consulente come facilitatore)
- [] La velocity è calcolabile e comunicabile al management
- [] Il DoD include almeno 1 gate di qualità automatico (test o build)
- [] Il tech debt è nel backlog e viene processato regolarmente
- [] DORA metrics mostrano miglioramento rispetto al mese 1
- [] Esiste una documentazione minima del processo (wiki o Notion page)
- [] Il team ha un piano per le prossime 8 settimane senza supporto esterno

3.4 Gestire la resistenza del middle management

Il middle management italiano è il più difficile da gestire in un programma Agile. Ha due paure fondamentali: perdere visibilità (non sa più "dove siamo") e perdere controllo (non può più assegnare task direttamente).

Script NLP per le 3 situazioni tipiche

Situazione 1 — Il manager che vuole un report giornaliero di avanzamento

Manager: "Ho bisogno di sapere ogni giorno quanto avete fatto."

Script: "Capisco perfettamente. Il vostro board [mostrare la board] mostra in tempo reale lo stato di ogni task. Questa settimana stiamo consegnando X, Y, Z. Il venerdì ho già pianificato un update di 15 minuti per voi. Vi va bene così, o preferite che aggiungiamo un breve report scritto ogni giovedì sera?"

Cosa fa questo script: soddisfa il bisogno (visibilità), educa allo strumento (board), propone un formato che non bypassa il processo.

Situazione 2 — Il manager che inserisce task urgenti a sprint iniziato

Manager: "Ho bisogno che questa cosa venga fatta entro giovedì, è urgente."

Script: "Nessun problema. Per farlo entro giovedì dobbiamo capire cosa spostiamo. Guardiamo insieme la board — avete 4 giorni, il team ha già 3 task in corso. Quale di queste tre volete rallentare per fare spazio a questa urgenza?"

Cosa fa questo script: non dice di no, trasferisce la decisione del trade-off al manager, rende visibile il costo dell'urgenza.

Situazione 3 — Il manager che non partecipa alla Sprint Review

Manager: "Non ho tempo per le review ogni due settimane."

Script: "La Sprint Review è 45 minuti ogni 2 settimane. È il momento in cui il team vi mostra cosa ha costruito e voi date feedback che cambia la direzione. Senza questo feedback, costruiamo in modo autonomo per 2 settimane — che va bene, ma poi il rischio è che arriviamo alla consegna finale con sorprese. Ci tenete che ci siano sorprese, o preferite 45 minuti ogni 2 settimane?"

3.5 KPI di successo del programma

Metriche baseline → target a 90 giorni

Metrica	Baseline tipica (PMI livello 1-2)	Target a 90 giorni	Come misurarla
Velocity (punti/sprint)	Non misurata o instabile	Stabile $\pm 20\%$ per 3 sprint consecutivi	Story point completati per sprint
Lead time	3-6 settimane	< 2 settimane	Jira: data creazione → data chiusura
Cycle time	5-10 giorni	< 3 giorni	Jira: data "In Progress" → data "Done"
Deployment frequency	Mensile o meno	Settimanale o bi-settimanale	Numero deploy in produzione al mese
Retrospective action completion	0-20%	> 60%	Azioni tracciate vs completate
Bug rate in produzione	Non tracciato	Trend decrescente (30-day baseline)	Ticket di bug aperti
Sprint goal achievement	Non misurato	> 70% degli sprint completano lo Sprint Goal	Sprint goal raggiunto sì/no per sprint

PARTE 4 — DevOps Foundation per PMI (senza budget enterprise)

Il principio guida: non vendere al cliente il sogno di Kubernetes con GitOps e service mesh. Vendi il minimo vitale che funziona. Un CI/CD semplice ma affidabile vale 10 volte più di un'architettura enterprise non mantenuta.

4.1 Tool stack cost-effective per PMI

Version control

GitLab Free (gitlab.com): gratis fino a 5 utenti con CI/CD inclusa. Scelta principale per team che vogliono tutto in un unico tool.

GitHub Free: team illimitati, Actions incluse, ma il piano Free ha limitazioni sui minuti di CI per repo privati (2.000 min/mese — sufficiente per team piccoli).

Raccomandazione: GitLab per chi parte da zero, GitHub se il team ha già familiarità.

CI/CD

GitLab CI: incluso nel piano Free. Sufficiente per il 90% delle PMI. Pipeline configurate via `.gitlab-ci.yml`.

GitHub Actions: incluso nel piano Free. Ottimo ecosistema di action pre-costruite. Configurazione via `.github/workflows/`.

Railway (railway.app): per deploy di applicazioni web senza bisogno di server. Piano Hobby gratuito, Piano Team da \$20/mese. Ideale per startup early stage.

Render (render.com): alternativa a Railway. Auto-deploy da GitHub/GitLab. Free tier disponibile.

Container

Docker + Docker Compose: obbligatorio. È lo standard. Non Kubernetes per team piccoli — il costo operativo è sproporzionato senza un DevOps dedicato.

Quando considerare Kubernetes: quando il team ha almeno 1 persona dedicata alla gestione infrastruttura, il prodotto ha requisiti di scalabilità eterogenei per componente, o si opera su cloud con supporto gestito (EKS, GKE, AKS).

Monitoring

Grafana Cloud Free tier: include 50GB di log, 10K metriche, 50GB trace per mese. Sufficiente per team piccoli.

UptimeRobot (uptimerobot.com): monitoring uptime gratuito per 50 monitor. Alert via email, Slack, Teams. Prima cosa da configurare su ogni produzione.

Sentry Free: error tracking per app web/mobile. 5.000 errori/mese gratuiti. Deve essere installato in ogni applicazione che va in produzione.

Testing

- JavaScript/TypeScript: Jest
- Python: pytest
- Java: JUnit 5 + Mockito

- .NET: xUnit + Moq
- PHP: PHPUnit

Project management

- Linear (linear.app): interface moderna, veloce, design orientato ai developer. Piano Free per team piccoli.
- Jira Free: fino a 10 utenti. Più complesso ma più diffuso. Scegliere se il cliente ha già familiarità o si integra con altri tool Atlassian.
- Notion: per team che vogliono unire task management, documentazione e wiki in un unico tool.

4.2 Come configurare la prima pipeline in 4 ore

Questo è il workshop pratico da fare nel mese 2 del programma. L'obiettivo è che al termine della giornata il team abbia una pipeline funzionante.

Prerequisiti (da verificare prima del workshop)

- [] Il codice è su GitLab o GitHub
- [] C'è almeno un test automatico (anche uno solo)
- [] Esiste un ambiente di staging (anche un server VPS da €5/mese su Hetzner o DigitalOcean)
- [] Il deploy manuale attuale è documentato (anche solo "copio i file via SFTP")

Step 1 — Configurare il repo (30 min)

Se non esiste ancora, creare branch strategy semplificata (vedi sezione 4.3).

Struttura minima del repo:

```
progetto/
├── src/
├── tests/
├── .gitlab-ci.yml (o .github/workflows/ci.yml)
├── Dockerfile
├── docker-compose.yml
└── README.md
```

Step 2 — Scrivere il Dockerfile (30 min)

Esempio per applicazione Node.js:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY src/ ./src/
EXPOSE 3000
CMD ["node", "src/index.js"]
```

Esempio per applicazione Python:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ ./src/
EXPOSE 8000
CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Step 3 — Configurare la pipeline CI (60 min)

Pipeline GitLab CI minima (.gitlab-ci.yml):

```
stages:
  - test
  - build
  - deploy

variables:
  DOCKER_IMAGE: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA

test:
  stage: test
  image: node:20-alpine
  script:
    - npm ci
    - npm test
  coverage: '/Coverage: \d+\.\d+/'

build:
  stage: build
  image: docker:24
  services:
    - docker:24-dind
  script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
    - docker build -t $DOCKER_IMAGE .
    - docker push $DOCKER_IMAGE
  only:
    - main
    - develop

deploy-staging:
  stage: deploy
  script:
    - ssh deploy@staging-server "docker pull $DOCKER_IMAGE && docker-compose up -d"
  environment:
    name: staging
  only:
    - develop

deploy-production:
  stage: deploy
  script:
    - ssh deploy@prod-server "docker pull $DOCKER_IMAGE && docker-compose up -d"
  environment:
    name: production
  when: manual
  only:
    - main
```

Pipeline GitHub Actions equivalente (.github/workflows/ci.yml):

```

name: CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: npm ci
      - run: npm test

  build-and-deploy:
    needs: test
    if: github.ref == 'refs/heads/main' || github.ref == 'refs/heads/develop'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build Docker image
        run: docker build -t ${{ secrets.REGISTRY }}/${{ github.repository }}:${{ github.sha }} .
      - name: Push to registry
        run: |
          echo ${{ secrets.REGISTRY_PASSWORD }} | docker login -u ${{ secrets.REGISTRY_USER }} --password-stdin ${{ secrets.REGISTRY }}
          docker push ${{ secrets.REGISTRY }}/${{ github.repository }}:${{ github.sha }}
      - name: Deploy to staging
        if: github.ref == 'refs/heads/develop'
        run: |
          ssh ${{ secrets.STAGING_USER }}@${{ secrets.STAGING_HOST }} \
            "docker pull ${{ secrets.REGISTRY }}/${{ github.repository }}:${{ github.sha }} && docker-compose up -d"

```

Step 4 — Configurare il monitoring (60 min)

- UptimeRobot: creare account, aggiungere monitor HTTP per ogni URL di produzione. Alert su Slack o email.
- Sentry: aggiungere SDK all'applicazione. Collegare a Slack per alert su errori nuovi.
- Grafana Cloud: per ora solo log aggregation. Aggiungere il Loki client all'applicazione per inviare i log.

Step 5 — Test della pipeline (60 min)

- Fare un commit su develop con un test che passa → verificare che la pipeline verdi
- Fare un commit con un test che fallisce → verificare che il deploy non avvenga
- Fare un deploy manuale su staging → verificare che il servizio sia raggiungibile
- Verificare che UptimeRobot invii alert se il servizio viene fermato manualmente

Deliverable del workshop

Al termine del workshop il team deve avere:

- Pipeline CI che esegue test ad ogni push
- Build Docker automatizzata
- Deploy automatico su staging dal branch develop
- Deploy manuale su produzione dal branch main
- UptimeRobot configurato
- Sentry configurato

4.3 Branching strategy semplificata per team piccoli

Non usare GitFlow pieno per team piccoli. È sovra-ingegnerizzato per team sotto 8 persone.

Git Flow semplificato (2-branch model)

```

main    ←→ produzione (deploy manuale, tag per ogni release)
↑
develop ←→ staging (deploy automatico ad ogni push)
↑
feature/xxx ←→ branch per ogni feature/bugfix (merge in develop, poi eliminare)

```

Regole:

- Non fare commit direttamente su main o develop
- Ogni feature ha il suo branch: feature/TICKET-123-nome-feature
- Merge request/Pull request obbligatoria per entrare in develop
- Tag semantico per ogni release in produzione: v1.2.3

Naming convention branch

- Feature: feature/TICKET-NNN-breve-descrizione
- Bugfix: bugfix/TICKET-NNN-breve-descrizione
- Hotfix (su main direttamente): hotfix/TICKET-NNN-breve-descrizione

Commit message convention (semplificata)

tipo(scope): descrizione breve

Corpo opzionale con più dettagli.

TICKET-NNN

Tipi: feat, fix, refactor, test, docs, chore, ci

Esempio: feat(auth): aggiunge autenticazione JWT per API esterna

4.4 Definition of Done con gate di qualità automatici

La DoD deve essere verificata automaticamente dove possibile. Questo elimina il problema della DoD "teorica" che nessuno rispetta davvero.

DoD minima per PMI (inserita come check nella pipeline CI)

```
# In .gitlab-ci.yml o GitHub Actions
quality-gate:
  stage: test
  script:
    # 1. Build senza errori
    - npm run build
    # 2. Linting
    - npm run lint
    # 3. Unit test con coverage minimo
    - npm test -- --coverage --coverageThreshold='{\"global\":{\"branches\":60,\"functions\":60,\"lines\":60}}'
    # 4. Security scan delle dipendenze
    - npm audit --audit-level=high
```

DoD per contesti diversi

Startup con legacy limitato:

- ☐ Il codice compila senza errori
- ☐ Almeno 1 test che copre la logica principale della feature
- ☐ Code review approvata da almeno 1 membro del team
- ☐ Funzionante in staging
- ☐ PO ha validato in staging

PMI con codebase esistente:

- ☐ Il codice compila senza errori
- ☐ Test di regressione esistenti passano
- ☐ Coverage non peggiora rispetto alla baseline
- ☐ Code review approvata
- ☐ Nessun secret hardcoded (verifica con git-secrets o trufflehog)
- ☐ Funzionante in staging
- ☐ Documentazione aggiornata se cambia comportamento pubblico

Team remoto:

- ☐ Tutti i criteri del contesto appropriato sopra
- ☐ Demo video (Loom) inviata al canale team
- ☐ Screenshot/video della feature incluso nella Merge Request

PARTE 5 — DORA Metrics per Team Piccoli

Le DORA metrics (ricerca Google DORA 2019, ora Accelerate State of DevOps) sono le 4 metriche che misurano la performance dei team di engineering. Sono il linguaggio più efficace per comunicare il ROI della consulenza al management.

5.1 Le 4 metriche adattate per PMI

Metrica 1 — Deployment Frequency (DF)

Definizione: quante volte deployate in produzione in una settimana/mese.

Benchmark DORA:

- Elite: più volte al giorno
- High: settimanale o più volte a settimana
- Medium: mensile
- Low: meno di mensile o semestrale

Benchmark realistico per PMI italiane al termine del programma:

- Punto di partenza tipico: mensile o meno
- Target a 90 giorni: settimanale o bi-settimanale

Come misurarla senza dashboard enterprise:

Google Sheet con una riga per ogni deploy. Colonne: Data, Componente, Versione, Chi ha fatto il deploy, Problema post-deploy (sì/no).

Contare le righe per settimana = DF.

Perché conta per il cliente: "Ogni volta che deployate, consegnate valore al cliente. Se deployate una volta al mese, il cliente riceve il suo miglioramento tra 4 e 8 settimane dopo la richiesta. Se deployate ogni settimana, il ciclo di feedback si dimezza."

Metrica 2 — Lead Time for Changes (LT)

Definizione: dal commit al deploy in produzione, quanto tempo passa?

Benchmark DORA:

- Elite: meno di 1 ora
- High: da 1 giorno a 1 settimana
- Medium: da 1 settimana a 1 mese
- Low: più di 1 mese

Benchmark realistico per PMI italiane al termine del programma:

- Punto di partenza tipico: 2-4 settimane
- Target a 90 giorni: 3-7 giorni

Come misurarla: in Jira o Linear, data del primo commit (o della card "In Progress") vs data del deploy in produzione. Calcolare la media su 10 deploy consecutivi.

Metrica 3 — Change Failure Rate (CFR)

Definizione: percentuale di deploy che richiedono hotfix, rollback o intervento urgente entro 48 ore.

Benchmark DORA:

- Elite: 0-15%
- High: 16-30%
- Low/Medium: >30%

Come misurarla: nel Google Sheet dei deploy, la colonna "Problema post-deploy" diventa il numeratore. $CFR = \text{deploy con problemi} / \text{deploy totali}$.

Perché conta: "Se ogni 4 deploy uno richiede hotfix urgente, state spendendo il 25% della vostra capacità a correggere quello che avete appena rilasciato. Con un CI/CD solido, questa cifra scende sotto il 15%."

Metrica 4 — Mean Time to Recovery (MTTR)

Definizione: quanto tempo passa dal momento in cui scoprite un problema in produzione al momento in cui è risolto.

Benchmark DORA:

- Elite: meno di 1 ora
- High: meno di 1 giorno
- Medium/Low: più di 1 giorno

Come misurarla: tenere un incident log. Anche semplice: data/ora scoperta problema, data/ora risolto. Media su tutti gli incident.

5.2 Come usare le metriche per mostrare il ROI

Le metriche devono essere tradotte in linguaggio business. Non mostrare al CEO un grafico del Lead Time senza contestualizzarlo.

Formula per comunicare il ROI:

"Prima del programma: [metrica baseline]. Dopo 90 giorni: [metrica attuale]. Questo significa che [impatto business]."

Esempi concreti:

"Prima deployavate ogni 3-4 settimane. Adesso ogni settimana. Questo significa che ogni richiesta del cliente arriva in produzione 2-3 settimane prima rispetto a prima."

"Il vostro Change Failure Rate è sceso dal 40% al 15%. Significa che ogni 10 deploy, invece di 4 hotfix urgenti ne avete 1-2. Il tempo liberato è direttamente reinvestito in nuove feature."

"Il Lead Time è sceso da 3 settimane a 5 giorni. Se avete 3 clienti che aspettano feature, ognuno di loro la riceve quasi 2 settimane prima rispetto a prima."

5.3 Report mensile DORA per il cliente (formato)

REPORT MENSILE — [Nome Cliente]
Mese: [Mese/Anno]
Consulente: Elios Scoglio

METRICHE DORA

Metrica	Baseline	Mese precedente	Questo mese	Trend
Deployment Frequency	X/mese	Y/mese	Z/mese	↑↓→
Lead Time for Changes	X giorni	Y giorni	Z giorni	↑↓→
Change Failure Rate	X%	Y%	Z%	↑↓→
Mean Time to Recovery	X ore	Y ore	Z ore	↑↓→

TOP 3 AZIONI DEL MESE

- 1. [Azione completata + risultato]
- 2. [Azione completata + risultato]
- 3. [Azione completata + risultato]

PROSSIMO SPRINT — FOCUS
[Sprint Goal del prossimo sprint]

IMPEDIMENTI APERTI
[Lista con owner e data prevista di risoluzione]

NOTE
[Osservazioni qualitative del mese]

PARTE 6 — Gestire la Resistenza e le Obiezioni

Questa parte è dove la formazione NLP entra in gioco. Le obiezioni all'Agile in Italia sono prevedibili. Prepararsi con script specifici elimina l'improvvisazione e aumenta il tasso di chiusura.

6.1 Le 7 obiezioni più comuni

Obiezione 1 — "Noi siamo diversi, Agile non funziona per noi"

Questa è l'obiezione più comune e più universale. Ogni azienda si sente unica.

Risposta NLP:

"In effetti ogni azienda è diversa. Per questo non applico Agile 'standard' — applico i principi Agile adattati al vostro contesto specifico. Nelle ultime [X] aziende che ho accompagnato, ognuna aveva le sue peculiarità. Quello che non cambia è il meccanismo: feedback rapidi, iterazioni brevi, visibilità sul lavoro in corso. Possiamo parlare di cosa rende il vostro contesto specifico e vedere insieme come adattarlo?"

Tecnica: aggancia "siamo diversi" come premessa vera, poi sposta il focus sull'adattamento. Non contraddi, co-costruisci.

Obiezione 2 — "Abbiamo già provato Scrum e non ha funzionato"

Questa obiezione nasconde quasi sempre una storia specifica. Prima di rispondere, fare domande.

"Interessante. Quando lo avete provato, e cosa è successo esattamente?"

Ascolto attivo. Le risposte tipiche rivelano: "il nostro PM non ha cambiato modo di lavorare", "abbiamo fatto le riunioni ma non abbiamo cambiato come scriviamo le storie", "dopo 2 mesi abbiamo smesso perché era faticoso".

Risposta dopo l'ascolto:

"Quello che mi state descrivendo è esattamente il problema più comune: si adottano le cerimonie di Scrum senza cambiare i pattern sottostanti (come vengono definiti i requisiti, come vengono prese le decisioni, come viene gestita l'urgenza). Quello che faccio io è partire da quei pattern, non dalle cerimonie. Se volete, il primo step è un assessment di un giorno dove vediamo insieme cosa ha fermato il processo la volta precedente."

Tecnica: trasforma l'esperienza passata in una diagnosi, non in una condanna. La colpa non è dello Scrum — è dell'implementazione incompleta. E c'è una soluzione.

Obiezione 3 — "Non abbiamo tempo per tutte queste riunioni"

Questa obiezione è quasi sempre legittima. Il team è già sovraccarico. Aggiungere cerimonie senza ridurre altro crea resistenza.

Risposta:

"Concordo al 100%. Le cerimonie Agile male implementate sono un costo, non un valore. Facciamo un calcolo insieme: in un team di 5 persone, un Daily di 30 minuti che non produce nulla costa 2,5 ore di team a settimana. Un Daily di 10 minuti che rimuove un impedimento ogni 2 giorni libera ore di lavoro bloccato. Il mio obiettivo non è aggiungere riunioni — è sostituire le riunioni lunghe e improvvisate con momenti corti e strutturati. Alla fine del primo mese, la maggior parte

dei team mi dice che ha meno riunioni di prima, non di più."

Tecnica: usa il calcolo concreto (2,5 ore) per rendere visibile il costo attuale. Poi ri-framing: non si aggiungono riunioni, si sostituiscono.

Obiezione 4 — "Il mio team non è disciplinato abbastanza per Agile"

Questa obiezione nasconde spesso una visione del management come "guardiano della disciplina". Il titolare/manager non si fida del team.

Risposta:

"La disciplina è il risultato del processo, non il prerequisito. Un team che non ha mai avuto visibilità chiara su quello che deve fare, scadenze realistiche e feedback continuo non può essere disciplinato — manca gli strumenti. L'Agile introduce proprio quegli strumenti. Nella mia esperienza, il team che 'non è disciplinato' di solito manca di chiarezza sui requisiti, di feedback tempestivo sugli errori, o di autonomia nelle decisioni. Quale di questi tre pensate sia il problema principale nel vostro caso?"

Tecnica: ri-framing della "mancanza di disciplina" come carenza di sistema, non di persone. Termina con una domanda che porta il management a auto-diagnosticare.

Obiezione 5 — "Costa troppo"

Il prezzo del programma sembra alto quando non è stato ancora stabilito il valore.

Risposta (solo dopo aver fatto l'assessment):

"Prima di rispondere sulla parte economica, permettetemi di quantificare il costo attuale. Sulla base dell'assessment: il vostro lead time attuale è [X settimane]. Ogni settimana di ritardo sulla consegna ha un costo diretto di [Y euro per sviluppatore × N settimane]. Il vostro Change Failure Rate di [Z%] significa [N hotfix al mese] × [M ore di sviluppatore per hotfix] = [K ore/mese bruciate su emergenze]. Solo su questi due driver, il costo attuale del vostro processo è circa [stima] al mese. Il programma a [€X/mese] si ripaga in [N mesi] se miglioriamo anche solo del 30% su questi driver. Volete procedere con questa logica?"

Tecnica: value-based selling. Non difendere il prezzo — mostrare il ROI. Questo richiede di aver fatto l'assessment prima di offrire il programma.

Obiezione 6 — "I nostri clienti non capirebbero le sprint review"

Obiezione comune nelle software house che lavorano su commessa con clienti non tech.

Risposta:

"Non serve che il cliente capisca Scrum. Serve che il cliente veda il progresso regolarmente e possa dare feedback prima che sia troppo tardi. La Sprint Review non è una riunione tecnica — è una demo del prodotto in 30-45 minuti. Quello che il cliente vede è: 'ogni 2 settimane mi mostrano quello che hanno costruito, posso dire se va bene o se devo cambiare direzione'. Questo è esattamente quello che ogni cliente vuole. I clienti che perdono la testa sui progetti software sono quelli che non vedono niente per 3 mesi e poi scoprono che non è quello che volevano."

Tecnica: tradurre la cerimonia in beneficio cliente. Il cliente non ha bisogno di sapere che si chiama Sprint Review.

Obiezione 7 — "Abbiamo già un PM, perché ci serve un consulente?"

Risposta:

"Il vostro PM gestisce il progetto. Io aiuto il team a lavorare meglio come sistema. Sono ruoli complementari, non concorrenti. Il PM sa cosa deve essere fatto e quando. Io lavoro sul come — sulle pratiche, sul processo, sulla cultura di miglioramento continuo. Nella mia esperienza, i PM che lavorano con me finiscono per avere meno fire-fighting e più tempo per la pianificazione strategica. Possiamo fare una call insieme ai tre — voi, il PM e io — per capire come strutturare la collaborazione?"

Tecnica: elimina la percezione di concorrenza, crea alleanza con il PM, proponi un passo concreto (call a tre).

6.2 Come gestire il titolare che vuole "controllo totale"

Il titolare italiano ha spesso una sindrome di controllo radicata nell'esperienza di aziende piccole dove tutto dipendeva da lui. Non è irrazionale — ha funzionato per anni.

Il bisogno sottostante: visibilità e certezza. Non vuole controllare i task per il gusto di controllare — vuole sapere che le cose stanno andando nella direzione giusta.

Strategia: dare più visibilità, non meno. Il board Kanban o la board Scrum aperta e condivisa con il titolare è il tool più potente. Non un report periodico — una finestra sempre aperta.

Script per la prima conversazione:

"Capisco perfettamente il vostro bisogno di sapere dove si trovano le cose in ogni momento. Una cosa che faremo subito è rendere il board visibile a voi in tempo reale — potete guardare lo stato di ogni task quando volete, senza dover chiedere a nessuno. In più ogni venerdì ho un update di 15 minuti con voi dove vi mostro i numeri della settimana. Quello che vi chiedo è di non interrompere il team durante la settimana per aggiornamenti — se avete domande, annotarle e portarle al venerdì. Questo riduce le interruzioni del team (che costano produttività) e vi garantisce informazioni strutturate invece di frammenti."

Script per quando bypassa il processo:

"Ho visto che ieri avete assegnato direttamente a [nome] la task X. Capisco l'urgenza. Il problema è che [nome] aveva già 3 task in corso — aggiungerne una senza rimuovere le altre crea sovraccarico e rallenta tutto. La prossima volta che c'è un'urgenza, fatemi sapere subito — passo io sulla board con voi in 5 minuti e insieme decidiamo cosa spostare. Così l'urgenza viene gestita senza impatto sul resto."

6.3 Come gestire il dev senior che odia le cerimonie

Il dev senior che ha 15 anni di esperienza e odia "le riunioni inutili" non ha torto sulla diagnosi — ha torto sulla cura. Le cerimonie male implementate sono inutili. Quelle ben implementate no.

Non forzare mai il dev senior. Quello che funziona è:

- Riconoscere esplicitamente la sua expertise tecnica
- Chiedere il suo input sulle cerimonie: "Come le renderesti meno costose?"
- Dargli un ruolo che sente di ownership: ad esempio, facilitare le retrospettive tech

Script:

"Ho notato che il Daily vi sembra uno spreco di tempo. Vorrei capire meglio — cosa cambiereste? Ho già qualche idea su come renderlo più efficace, ma preferirei sentire prima la vostra."

Quasi sempre il dev senior propone: "tenerlo più corto", "non farlo stare in piedi come robot", "parlare solo di blocchi reali, non di update di stato". Queste sono tutte proposte valide. Implementarle e attribuire il cambiamento al dev senior. Questo crea ownership e riduce la resistenza.

6.4 Segnali di sabotaggio silenzioso

Il sabotaggio silenzioso è più pericoloso dell'opposizione aperta. Questi sono i segnali da riconoscere entro il primo mese.

Segnale | Interpretazione | Intervento

Il board non viene aggiornato dopo il primo entusiasmo | Il team non vede valore nell'aggiornamento | Capire cosa blocca: overhead? Non sanno come

Le retrospettive producono sempre le stesse azioni che non vengono mai chiuse | Disengagement o cinismo ("tanto non cambia niente") | Ridurre le a

Il Daily diventa un report asincrono su Slack senza interazione | Il team non vede il valore dell'incontro sincrono | Capire il vero bisogno: trop

Le stime vengono sistematicamente sbagliate nella stessa direzione | Gaming del sistema o planning pressure | Verificare se c'è pressione esterna

Il manager "giusto per oggi" bypassa il board | Il management non ha fatto il commitment | Conversazione diretta con il titolare. Il programma fun

Il dev senior fa le task ma non le segna come "Done" | Non si riconosce nel processo | Capire la causa: orgoglio? Non sa come? Detesta il tool?

Principio generale: il sabotaggio silenzioso è sempre un segnale di sistema, non di persona. Prima di giudicare, diagnosticare.

PARTE 7 — Template e Strumenti

7.1 Sprint Charter

SPRINT CHARTER

Progetto: [Nome Progetto]
Sprint numero: [N]
Date: [inizio] → [fine]
Compilato da: [nome]

CHI

Product Owner: [nome]
Scrum Master (o facilitatore): [nome]
Team members: [lista]
Stakeholder principali: [lista]

SPRINT GOAL

[Una frase che descrive il valore che questo sprint consegna. Esempio:
"Al termine di questo sprint, il cliente può completare l'ordine online senza dover chiamare il supporto per le opzioni di spedizione."]

CAPACITY

Giorni lavorativi disponibili nel periodo: [N]
Giorni di ferie/assenze previste: [N]
Capacity effettiva: [N] giorni × [ore/giorno] = [N] ore totali

ITEMS SELEZIONATI

ID	User Story	Story Points	Owner	Note
---	---	---	---	---

Totale punti: [N]

DEFINITION OF DONE (per questo sprint)

- [] Code review completata
- [] Test automatici scritti e passanti
- [] Funzionante in staging
- [] PO ha approvato in staging
- [] [Item specifico del progetto]

RISCHI IDENTIFICATI

Rischio	Probabilità	Impatto	Mitigazione
---	---	---	---

NOTE

[Qualsiasi elemento contestuale rilevante per questo sprint]

7.2 Definition of Done per diversi contesti

DoD — Startup con prodotto greenfield

DEFINITION OF DONE — [Nome Startup]

Aggiornata: [data]

Una User Story è DONE quando:

1. Il codice è scritto e passa il build
2. Esiste almeno 1 test che verifica il comportamento principale della feature
3. Il linting è pulito (nessun warning bloccante)
4. La Merge Request è stata approvata da almeno 1 altro membro del team
5. La feature è deployata in staging
6. Il Product Owner ha visto e approvato la feature in staging
7. Non sono stati introdotti regressioni sulle feature esistenti
8. La documentazione della API (se applicabile) è aggiornata

DoD — PMI con codebase legacy**DEFINITION OF DONE — [Nome Azienda]**

Aggiornata: [data]

Una User Story è DONE quando:

1. Il codice è scritto e compila senza errori
2. Tutti i test esistenti passano ancora (nessuna regressione)
3. Se la feature tocca logica critica, è stato aggiunto almeno 1 test nuovo
4. La coverage del modulo modificato non è diminuita
5. La Merge Request è approvata da almeno 1 persona con conoscenza del modulo
6. Nessun secret hardcoded nel codice (verifica automatica in pipeline)
7. La feature è funzionante in staging
8. Il PO o referente del cliente ha visto la feature in staging
9. Il ticket Jira è aggiornato con i dettagli dell'implementazione
10. Se ci sono cambiamenti al database, lo script di migrazione è incluso e testato

DoD — Team remoto multi-fuso orario**DEFINITION OF DONE — [Nome Team]**

Aggiornata: [data]

Una User Story è DONE quando:

1. [Tutti i criteri tecnici del contesto appropriato]
2. Un video Loom (max 3 min) mostra la feature funzionante
3. Il video è postato nel canale #reviews del team
4. Il PO ha commentato/approvato il video entro 24 ore
5. La Merge Request include screenshot o video della feature
6. Tutte le domande nella MR sono state chiuse (nessun commento pending)

7.3 Retrospective Canvas — 4L

RETROSPETTIVA SPRINT [N]
Data: [data]
Facilitatore: [nome]
Partecipanti: [lista]

LIKED
(Cosa ci è piaciuto)

LEARNED
(Cosa abbiamo imparato)

LACKED
(Cosa è mancato)

LONGED FOR
(Cosa avremmo voluto avere)

AZIONI (max 3, con owner e scadenza)

#	Azione	Owner	Scadenza	Verificata (sprint N+1)
1				
2				
3				

STATO AZIONI SPRINT PRECEDENTE

Azione	Completata?	Note
[Azione da sprint N-1]	Si/No	

7.4 Velocity Dashboard — Google Sheets

Struttura del foglio "Velocity"

Sprint	Data Inizio	Data Fine	Punti Pianificati	Punti Completati	Sprint Goal Raggiunto	Note
Sprint 1					Si/No	
Sprint 2					Si/No	
Sprint 3					Si/No	

Struttura del foglio "DORA"

Data Deploy	Componente	Versione	Chi	Ambiente	Tempo dal Commit (ore)	Problema entro 48h?	Tempo Risoluzione (ore)	Note
Prod						Si/No		

Grafici da generare (Google Sheets)

- Velocity per sprint (grafico a barre) — mostra la stabilizzazione nel tempo
- Trend Lead Time (grafico a linee) — mostra la riduzione
- Change Failure Rate (percentuale mensile) — mostra il miglioramento
- Sprint Goal Achievement Rate (percentuale per mese) — mostra la maturità

7.5 Onboarding Checklist Cliente Agile (50 item)

A — Accessi e Strumenti (10 item)

- ☐ A1. Accesso al repo Git del progetto (GitLab/GitHub) — ruolo: Developer o Maintainer
- ☐ A2. Accesso al tool di project management (Jira/Linear/Trello) — ruolo: con permesso di modifica board
- ☐ A3. Accesso al canale di comunicazione principale (Slack/Teams)
- ☐ A4. Accesso alla pipeline CI/CD (visibilità dei log)
- ☐ A5. Accesso all'ambiente di staging (URL + credenziali)
- ☐ A6. Accesso alla dashboard di monitoring (Grafana/UptimeRobot)
- ☐ A7. Accesso al sistema di error tracking (Sentry)
- ☐ A8. Accesso alla documentazione esistente (Confluence/Notion/Wiki)
- ☐ A9. Invito al calendario delle cerimonie ricorrenti
- ☐ A10. Numero di telefono/contatto diretto del titolare e del PM

B — Persone Chiave (10 item)

- ☐ B1. Nome e ruolo del Product Owner identificato
- ☐ B2. Nome e ruolo di chi gestisce il backlog oggi (anche informalmente)
- ☐ B3. Dev senior tecnico di riferimento identificato
- ☐ B4. Eventuali stakeholder esterni che parteciperanno alle Sprint Review
- ☐ B5. Chi prende le decisioni di priorità quando il PO non è disponibile?
- ☐ B6. Chi ha accesso ai deploy in produzione (e relativo processo di approvazione)?
- ☐ B7. Referente IT/infrastruttura (se presente)
- ☐ B8. Chi gestisce i clienti (per le Sprint Review con clienti)
- ☐ B9. Chi è il "campione interno" Agile (la persona più motivata nel team)
- ☐ B10. Chi è la persona con la maggiore resistenza al cambiamento? (non da condividere — solo per me)

C — Contesto Tecnico (10 item)

- ☐ C1. Stack tecnologico documentato (linguaggi, framework, database)
- ☐ C2. Architettura attuale descritta (anche a voce + schema su carta)
- ☐ C3. Processo di deploy attuale documentato (step-by-step, anche se è tutto manuale)
- ☐ C4. Lista degli ambienti: dev, staging, produzione (URL + caratteristiche)
- ☐ C5. Test esistenti identificati (quanti, dove, tipo)
- ☐ C6. Lista delle dipendenze esterne critiche (API di terze parti, sistemi legacy)
- ☐ C7. Incidenti recenti documentati (ultimi 3 mesi: cosa, quando, quanto durato)
- ☐ C8. Lista del debito tecnico noto (anche se non strutturata)
- ☐ C9. Segreti/credenziali gestiti: sistema attuale (variabili d'ambiente? file .env? hardcoded? — verificare)
- ☐ C10. Backup: esiste? Quando è stato testato l'ultimo restore?

D — Processo Attuale (10 item)

- ☐ D1. Backlog attuale: dove vive, quanti item, come è organizzato
- ☐ D2. Processo di stima attuale (esiste? come funziona?)
- ☐ D3. Come vengono comunicate le richieste al team oggi?
- ☐ D4. Frequenza dei meeting attuali e loro scopo
- ☐ D5. Processo di code review (esiste? formale o informale?)
- ☐ D6. Metriche tracciate oggi (se esistono)
- ☐ D7. Ultimo retrospective o post-mortem fatto: quando, cosa ha prodotto
- ☐ D8. Processo di gestione delle urgenze: come arrivano, chi decide, come vengono gestite
- ☐ D9. Definition of Done attuale (esiste? è scritta? è rispettata?)
- ☐ D10. Release notes: esistono? come vengono comunicate ai clienti?

E — Obiettivi e Aspettative (10 item)

- [] E1. Obiettivo principale del programma: cosa deve essere diverso tra 3 mesi?
 - [] E2. Metriche di successo concordate con il titolare/management
 - [] E3. Eventuali scadenze esterne fisse nei prossimi 3 mesi (contratti, eventi, lancio prodotti)
 - [] E4. Budget per tool/infrastruttura: c'è? Quanto?
 - [] E5. Disponibilità del team per le cerimonie: orari, giorni con più vincoli
 - [] E6. Eventuali ferie/periodi di ridotta disponibilità nei prossimi 3 mesi
 - [] E7. Aspettative del team sul programma (raccolte nel kick-off)
 - [] E8. Paure/resistenze del team sul programma (raccolte individualmente)
 - [] E9. Cosa ha già provato il cliente in passato (e perché non ha funzionato)
 - [] E10. Referral/referenze: chi potrebbe beneficiare del programma nel network del cliente?
-

7.6 Report mensile cliente (1 pagina)

REPORT MENSILE AGILE & DEVOPS

Cliente: [Nome]
Mese: [Mese/Anno]
Consulente: Elios Scoglio

METRICHE DEL MESE

Metrica	Obiettivo	Attuale	Trend
Velocity (punti/sprint)	[target]	[valore]	↑↓→
Deployment Frequency	[target]	[valore]	↑↓→
Lead Time	[target]	[valore]	↑↓→
Change Failure Rate	[target]	[valore]	↑↓→
Sprint Goal Achievement	>70%	[valore%]	↑↓→

TOP 3 RISULTATI DEL MESE

- 1. [Risultato con dato numerico se possibile]
- 2. [Risultato con dato numerico se possibile]
- 3. [Risultato con dato numerico se possibile]

AZIONI CHIUSE (dalla scorsa settimana)

- [Azione] — [risultato]
- [Azione] — [risultato]

PROSSIMO SPRINT

Goal: [Sprint Goal]
Periodo: [data inizio] → [data fine]
Punti pianificati: [N]

IMPEDIMENTI APERTI

Impedimento	Owner	Data prevista risoluzione

NOTE E OSSERVAZIONI

[Max 3 righe. Solo il punto più rilevante del mese.]

PARTE 8 — Upsell e Continuità

8.1 Come trasformare un engagement Agile in consulenza continuativa

Il programma 90 giorni termina con un team più autonomo. Ma l'autonomia non significa che non c'è più valore nel supporto esterno. Il posizionamento corretto è: dopo i 90 giorni il team sa camminare da solo, ma può beneficiare di un "allenatore" mensile che lo aiuta a continuare a migliorare.

Prodotto di continuità: Sprint Coaching mensile

Prezzo: €2.000–€3.000/mese

Contenuto: 1 giornata al mese (presenza o remoto) composta da:

- Revisione delle metriche del mese
- Facilitazione della retrospettiva mensile estesa (60 min)
- Coaching 1:1 con il team leader/SM (60 min)
- Backlog review e prioritizzazione (60 min)
- Action items per il mese successivo

Come proporre il passaggio:

Non presentarlo come "continuazione della consulenza". Presentarlo come "manutenzione del sistema". Analogia efficace: "Un'auto nuova funziona benissimo. Ma un tagliando ogni anno garantisce che continui a farlo. Il mio ruolo nel mese 4 in poi è un tagliando mensile al vostro processo — 1 giorno, focus su quello che è scivolato, quello che si può migliorare e quello che si può ottimizzare."

Quando proporre la continuità: nella call finale del mese 3 (retrospettiva del programma). Non prima — il cliente deve aver già visto i risultati.

Script di chiusura-apertura:

"In questi 3 mesi avete fatto [risultati concreti con numeri]. Il team ha il processo, le metriche, gli strumenti. Quello che ho visto in molti programmi è che senza un momento di verifica esterno, nel giro di 2-3 mesi alcune buone pratiche iniziano a erodere — non per malafede, ma perché il lavoro quotidiano prende il sopravvento. Quello che propongo è un check mensile di 1 giorno: garantisce che il sistema resti sano e che continuiate a migliorare. I clienti che lo fanno dopo il programma migliorano ancora del 30-40% nei 6 mesi successivi. Quelli che non lo fanno tornano spesso al 60-70% del livello pre-programma entro un anno. Come volete procedere?"

8.2 Il percorso naturale di upsell

Il programma Agile apre naturalmente le porte a due percorsi di upsell ad alto valore.

Percorso A — Agile Assessment → 90-Day Program → Fractional CTO

Questo percorso funziona quando:

- L'azienda è in crescita e ha bisogno di guida strategica continua
- Emergono durante il programma problemi architetturali o di team che richiedono leadership tecnica
- Il titolare inizia a vedere il consulente come un partner strategico, non solo come formatore

Come si manifesta: durante il programma il cliente dice frasi come "vorremmo fare X ma non sappiamo come strutturarci" o "stiamo crescendo, dovremmo assumere un tecnico senior — come scegliamo?" o "abbiamo un'opportunità con un cliente grande, dobbiamo cambiare qualcosa nel modo in cui sviluppiamo?"

Risposta: "Quello che state descrivendo va oltre il processo Agile — è una questione di leadership tecnica e strategia. È esattamente il territorio del Fractional CTO. Se vi interessa, possiamo fare una call dedicata per capire se ha senso per voi."

Transizione economica: dal €2.000-€3.000/mese dello Sprint Coaching al €5.000-€10.000/mese del Fractional CTO. Il cliente ha già la fiducia costruita, il salto percettivo è molto più piccolo di una proposta a freddo.

Percorso B — Agile Assessment → 90-Day Program → DIGI / AI Adoption

Questo percorso funziona quando:

- Durante il programma emergono processi manuali ripetitivi che si potrebbero automatizzare
- Il team non usa AI tools nel lavoro quotidiano (editor, test generation, code review, documentazione)
- L'azienda vuole accelerare il time-to-market e la risposta ovvia è "fate fare di più alla macchina"

Come si manifesta: il team dice "aggiorniamo la documentazione a mano dopo ogni sprint — è un lavoro enorme", oppure "scriviamo i test a mano dopo il codice — spesso non li scriviamo perché ci vuole troppo tempo", oppure "fare code review su codebase grandi è lento".

Risposta: "Quello che state descrivendo è esattamente il territorio dell'AI Adoption per team di engineering. Non si tratta di comprare un tool — si tratta di cambiare il workflow in modo che l'AI amplifichi il vostro team. Ho un programma dedicato che parte da quello che avete già costruito con l'Agile. Vuoi che te ne parli?"

Transizione verso DIGI/AI Adoption: il track AI Adoption è posizionato come "il passo successivo naturale" dopo aver stabilizzato il processo. L'ordine è intenzionale: prima il processo funziona (Agile), poi si amplifica (AI).

8.3 Come posizionare il track Agile come gateway per DIGI e AI Adoption

Il track Agile è il punto di ingresso più basso nel portfolio di consulenza. Quasi ogni PMI italiana ha un bisogno di Agile — è tangibile, rapido, misurabile. Non richiede un titolare tech-savvy per capirne il valore.

Ma la vera posizione strategica del track Agile non è solo il suo valore intrinseco. È la porta che apre.

Il flywheel del portfolio

Assessment (€5K, 1 giorno)

- dimostra credibilità + svela i problemi reali
- porta naturalmente al 90-Day Program

90-Day Program (€4-8K/mese × 3 mesi)

- costruisce fiducia profonda
- rivela bisogni di leadership tecnica e AI
- porta a Sprint Coaching continuativo

Sprint Coaching (€2-3K/mese)

- relazione duratura, bassa intensità
- osservatorio privilegiato sull'evoluzione dell'azienda
- lead naturale per Fractional CTO o DIGI/AI

Fractional CTO o AI Adoption Program

- engagement ad alto valore (€5-15K/mese)
- ownership strategica
- porta a opportunità di lungo periodo

Come presentarlo al mercato (positioning statement)

Per il cliente: "Aiuto PMI italiane a consegnare software di qualità più velocemente, con un team che si auto-organizza e migliora nel tempo. Parto sempre da un assessment di un giorno per capire dove siete, poi costruiamo insieme un piano realistico."

Per il mercato: "Agile & DevOps per PMI" come specializzazione, non come commodity. Non "faccio formazione Scrum" — "trasformo il modo in cui il vostro team sviluppa software, con risultati misurabili in 90 giorni."

Pricing summary del portfolio Agile & DevOps**Prodotto | Prezzo | Durata | Output principale**

Agile Maturity Assessment	€5.000	1 giorno + report (48h)	Scorecard + roadmap prioritizzata
Workshop Agile/DevOps	€4.000–€6.000/giorno	1 giorno	Team trained + action plan
90-Day Agile Adoption Program	€4.000–€8.000/mese	3 mesi	Team autonomo + metriche baseline
Sprint Coaching (continuativo)	€2.000–€3.000/mese	Ongoing	Miglioramento continuo garantito
DevOps Foundation Workshop	€4.000	1 giorno	Pipeline CI/CD funzionante

Come qualificare un prospect per il track Agile

Prima di proporre il servizio, rispondere a queste domande. Se almeno 3 delle 5 sono vere, il prospect è qualificato.

- Il team sviluppa software (non solo lo usa)?
- C'è una pressione percepita su tempi, qualità o comunicazione interna?
- Il titolare o responsabile ha già sentito parlare di Agile (anche negativamente)?
- Il team ha almeno 3 persone dedicate allo sviluppo?
- L'azienda ha intenzione di continuare a sviluppare software nei prossimi 2+ anni?

Se tutte e 5 sono vere: proporre direttamente il Assessment.

Se 3-4 sono vere: proporre un call esplorativo di 30 minuti.

Se meno di 3: non è il momento giusto. Lasciare aperto il contatto per il futuro.

Note finali operative

Prima di ogni ingaggio Agile:

- Leggere la Parte 1 e preparare il protocollo di assessment specifico per il cliente
- Preparare la scorecard in anticipo (template in Excel o Notion)
- Avere pronti gli script di risposta alle obiezioni (Parte 6)

Durante il programma:

- Tenere un journal privato delle osservazioni su ogni persona del team (non condividerlo mai)
- Aggiornare le metriche DORA dopo ogni deploy, non a fine mese
- Non esitare a rallentare se il team è sopraffatto — è meglio fare meno e farlo bene

Segnali che il programma sta funzionando:

- Il team aggiorna il board senza che nessuno lo chieda
- Le cerimonie iniziano senza aspettare il consulente
- Il management smette di bypassare il processo
- Il team propone autonomamente miglioramenti nelle retrospettive
- Le metriche DORA mostrano trend positivi per 2+ sprint consecutivi

Segnali che il programma è a rischio:

- Il board è aggiornato solo il giorno prima del check-in
- Le retrospettive producono sempre le stesse azioni
- Il management continua a inserire urgenze senza trade-off
- Il team non distingue tra "Daily" e "status update al manager"
- Nessuna azione dalla retro viene completata

In questo caso: stop al programma normale. Call urgente con il titolare. Problema di sistema, non di processo. Diagnosi prima di procedere.

Manuale interno — Elios Scoglio

Versione 1.0 — Maggio 2026

Aggiornare dopo ogni nuovo ingaggio con le osservazioni sul campo.